
UNIT 4 FILE ORGANISATION IN DBMS

Structure	Page Nos.
4.0	Introduction
4.1	Objectives
4.2	Physical Database Design
4.3	Database Storage on HDD and SSD
4.4	File Organisations
4.4.1	Unordered Heap File Organisation
4.4.2	Sequential File Organisation
4.4.3	Indexed (Indexed Sequential) File Organisation
4.4.4	Hashed File Organisation
4.5	Indexes
4.6	Implementing Index Using Tree Structure
4.7	Multi-key File Organisations
4.7.1	Multiple Access Paths
4.7.2	Multi-list File Organisation
4.7.3	Inverted File Organisation
4.8	Importance of File Organisation in Databases
4.9	Summary
4.10	Solutions/Answers

4.0 INTRODUCTION

In earlier units, you studied the basic concepts of database management systems, entity relationship diagram and relational algebra. Databases are used to store information. Normally, the principal operations you need to perform on database are those relating to:

- Creation of data
- Retrieving the data using conditions
- Modifying
- Deleting some information, which we are sure is no longer useful or valid.

Database structures data as two-dimensional tables, which allows easy processing of these operations. However, as the size of databases are large, the databases are required to be stored on secondary memory of computers (such as hard disk or SSD). Therefore, the secondary storage systems of databases are mainly concerned with the following issues:

- Storing table or tables as files: A single table can be stored as a file or several tables can be put together as a cluster of related records called cluster file.
- Attributes of a table: In general, a table represents the data of one object, therefore, the attributes represent data of a specific object and may be required to be accessed by a specific operation. Thus, it may be a good idea to store all the attributes of an object together. Does the order of attributes in a file matter?
- It seems logical to store all the records of a table contiguously. But, how should such records be ordered? The ordering of records in primary storage does not matter, however, for the secondary storage a specific sequence may be desired. Such decisions may be taken by the database demonstrator and may relate to the performance of the database.
- In the cases of analytical queries, particular attributes are stored together, this approach is called column-oriented approach, this approach is beyond the scope of this Unit. You may refer to further readings for this approach.

This unit focuses on the file Organisation in DBMS, the access methods available and the system parameters associated with them. File Organisation is the way the files are arranged on the disk and access method is how the data can be retrieved based on the file organisation.

4.1 OBJECTIVES

After going through this unit, you should be able to:

- define storage of databases on hard disks and SSD;
 - discuss the implementation of various file Organisation techniques;
 - discuss the advantages and the limitation of the various file Organisation techniques;
 - describe various indexes used in database systems, and
 - define the multi-key file organisation.
-

4.2 PHYSICAL DATABASE DESIGN

The database design involves the process of logical design with the help of E-R diagram, normalisation, etc., followed by the physical design.

The Key issues in the Physical Database Design are:

- The purpose of physical database design is to translate the logical description of data into the technical specifications for storing and retrieving data for the DBMS.
- The goal is to create a design for storing data that will provide adequate performance and ensure database integrity, security and recoverability.

Some of the basic inputs required for Physical Database Design are:

- Normalised relations
- Attribute definitions
- Data usage: entered, retrieved, deleted, updated
- Requirements for security, backup, recovery, retention, integrity
- DBMS characteristics.
- Performance criteria such as response time requirement with respect to volume estimates.

However, for such data some of the Physical Database Design Decisions that are to be taken are:

- Optimising attribute data types.
- Modifying the logical design.
- Specifying the file Organisation.
- Choosing indexes.

Designing the attributes in the database

The following are the considerations one has to keep in mind while designing the attributes in the database.

- Choosing data type
- Coding, compression, encryption
- Controlling data integrity
- Default value
 - ❑ Range control
 - ❑ Null value control
 - ❑ Referential integrity
- Handling missing data

- ❑ Substitute an estimate of the missing value
- ❑ Trigger a report listing missing values
- ❑ In programs, ignore missing data unless the value is significant.

Physical Records

These are the records that are stored in the secondary storage devices. For a database relation, physical records are the group of fields stored in adjacent memory locations and retrieved together as a unit. Considering the page memory system, a data page is the amount of data read or written in one I/O operation to and from a secondary storage device to the memory and vice-versa. In this context we define a term blocking factor that is defined as the number of physical records per page.

The issues relating to the Design of the Physical Database Files

Physical File is a file as stored on the disk or SSD. The main issues relating to physical files are:

- Constructs to link two pieces of data:
 - ❑ Sequential storage.
 - ❑ Pointers.
- File Organisation: How are the files arranged on the disk?
- Access Method: How can the data be retrieved based on the file Organisation?

Let us see in the next section how the data is stored on the hard disks (HDD) or SSDs

4.3 DATABASE STORAGE ON HDD OR SSD

At this point, it is worthwhile to note the difference between the terms file Organisation and the access method. A file organisation refers to the organisation of the data records, which are part of a file, into a group of records that are placed in a block of secondary storage. The file organisation also entails the access structures and interlinking of records. An access method is the way - how the data can be retrieved based on the file Organisation.

Mostly the databases are stored persistently on HDD or SSD for the reasons given below:

- The databases being very large may not fit completely in the main memory.
- Data of the database is to be stored permanently using non-volatile storage.
- Primary storage is expensive, using secondary storage reduces the cost of the storage per unit.

Each hard drive is usually composed of a set of disk platters. Each disk platter has a layer of magnetic material deposited on its surface. The entire disk can contain a large amount of data, which is organised into smaller packages called BLOCKS (or pages). On most computers, one block is equivalent to 1 KB of data (= 1024 Bytes). A block is the smallest unit of data transfer between the hard disk and the processor of the computer. Each block therefore has a fixed, assigned, address. Typically, the computer processor will submit a read/write request, which includes the address of the block, and the address of RAM in the computer memory area called a buffer (or cache) where the data must be stored/taken from. The processor then reads and modifies the buffer data as required and, if required, writes the block back to the disk. Let us see how the tables of the database are stored on the hard disk.

A Solid-State Disk (SSD) is made up of flash memories. These SSD devices also provide similar block-oriented access to data, however, since they do not have physically moving components, they provide lower latency and lower access time than that of disks. Therefore, in the subsequent sections, we will provide details on the disk oriented approach of data storage.

Fixed and Variable Length Records

There are two basic ways of storing a record on the disks – Fixed Length records and Variable Length Records. In the fixed length records all the attributes are assigned equal space in terms of bytes, just like fixed length structure in C programming. Thus, each record will be of the same size. In such cases, only metadata can be used to identify different records and their attributes.

As far as variable length records are concerned, it may be noted that each record of a table may be of different length. This is because in some records, some attribute values may be 'null'; or some attributes may be of type *varchar*, which allows variable number of characters to be stored in an attribute, therefore each record may have a different length string as the value of this attribute. To store variable length records, it is necessary to use a character that marks the end of an attribute value. In addition, an end of record marker will also be needed, as one disk block may store several records. Therefore, each record is separated from the next, again by another special character, the record separator.

The next section discusses different types of file organisation that can be used to store the files on the disks.

4.4 FILE ORGANISATIONS

File organisation is used to organise the content of a file on a secondary storage device. A good file organisation should support efficient data access and update operations on the content of a file, which is stored on the secondary storage. Data files are organised so as to facilitate access to records and to ensure their efficient storage. A tradeoff between these two requirements generally exists: if rapid access is required, more storage space is required to make it possible. Selection of File Organisations is dependent on two factors, as shown below:

- Typical DBMS applications may need to access a small subset of the data of a database at any given time.
- Whenever a portion of the data is needed by the DBMS; it is located on disk, copied to memory for processing, and rewritten to disk if the data was modified.

A file of record is likely to be accessed and modified in a variety of ways, and different ways of arranging the records enable different operations over the file to be carried out efficiently. A DBMS supports several file Organisation techniques. The important task of the DBA is to choose a good Organisation for each file based on its type of use.

For a database system, you select a file organisation a suitable file organisation based on the parameters like number of keys used to access the files, the ratio of update and access operations, the type of storage technology etc. *Figure 4.1* uses access keys as the basis for classifying different file organisations. This classification will be used in this unit to describe various file organisations.

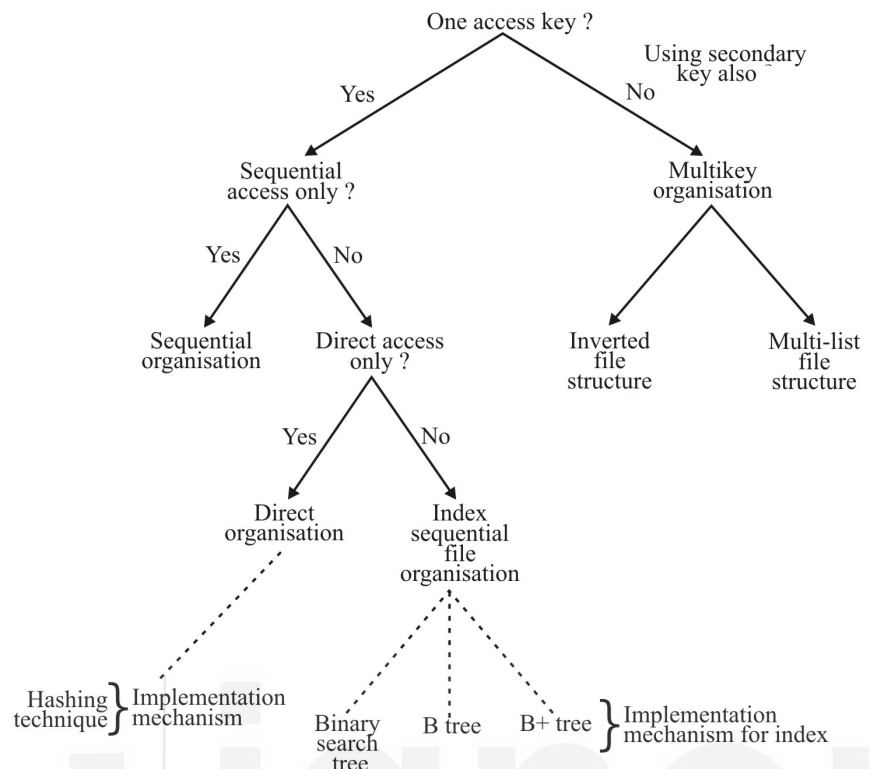


Figure 4.1: Different File Organisation Based on Access Keys

Let us discuss some of these techniques in more detail:

4.4.1 Unordered Heap File Organisation

Basically, these files are unordered files. It is the simplest and most basic type. These files consist of randomly ordered records. The records will have no particular ordering. The operations that you can perform on the records of a heap file are insert, retrieve and delete. The features of the heap file Organisation are:

- New records can be inserted in any empty space that can accommodate them.
- When old records are deleted, the occupied space becomes empty and available for any new insertion.
- If updated records grow, they may need to be relocated (moved) to a new empty space. Thus, this file organisation keeps a list of empty spaces.

Advantages of heap files

1. This is a simple file Organisation.
2. Insertion is somehow efficient.
3. Good for bulk-loading data into a table.
4. Best if file scans are common or insertions are frequent.

Disadvantages of heap files

1. Retrieval requires a linear search and is inefficient.
2. Deletion can result in unused space/need for reorganisation.

4.4.2 Sequential File Organisation

In the sequential Organisation, records of the file are stored in sequence (or consecutively) by the primary key field values. In this organisation, the records can be accessed in the order of its primary key. This kind of file Organisation works well for tasks in which nearly every record of the file is required to be accessed, such as the payroll system. Figure 4.2 shows this file organisation.

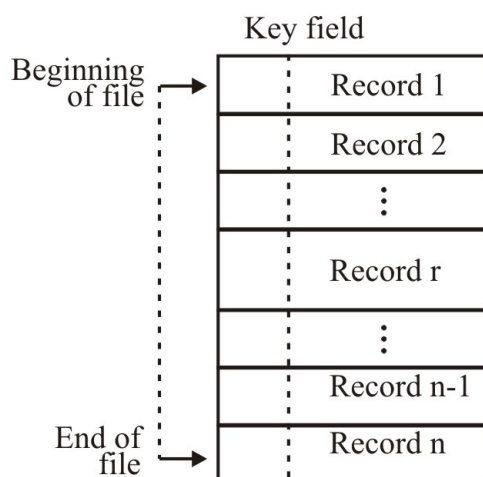


Figure 4.2: Structure of sequential file

A sequential file maintains the records in the logical sequence of its primary key values. Sequential files are inefficient for random access, however, are suitable for sequential access. A sequential file can be stored on devices like magnetic tape that allow sequential access.

On an average, to search a record in a sequential file would require looking into half of the records of the file. However, if a sequential file is stored on a disk (remember disks support direct access of its blocks) with keys stored separately from the rest of the record, then only those disk blocks are needed to be read that contains the desired record or records. This type of storage allows binary search on sequential file blocks, thus, enhancing the speed of access.

Updating a sequential file usually creates a new file so that the record sequence on the primary key is maintained. The update operation first copies the records till the record after which update is required into the new file and then the updated record is put followed by the remainder of records. Thus, the method of updating a sequential file automatically creates a backup copy. However, such update operations are very time consuming.

Addition of records in the sequential files are also handled in a similar manner to update operation. If a record is to be inserted at the last record of the file, it can be performed very easily. However, if a record is required to be inserted in between two records, then such insertion would require shifting down all the subsequent records in the file by one record space. In case of deletion of a record, all the subsequent records need to be shifted up by one record space.

Sequential file organisation is most suitable, if all the records of a file are to be processed in a sequence. For example, processing the monthly payroll of all the employees of an organisation, will require processing of all the employees records sequentially. However, a single update is expensive as a new file must be created, therefore, to reduce the cost per update, all update requests are stored in a single update file, which is sorted in the order of the sequential file ordering key. The file containing the updates is sometimes referred to as a transaction file and is used to update the sequential file in a single processing cycle.

This process is called the batch mode of updating. In this mode, each record of the master sequential file is checked for one or more possible updates by comparing with the update information of the transaction file. The records are written to a new master file in a sequential manner. A record that requires multiple updates is written only when all the updates have been performed on the record. A record that is to be deleted

is not written to a new master file. Thus, a new updated master file will be created from the transaction file and the old master file.

Thus, update, insertion and deletion of records in a sequential file require a new file creation. Can we reduce creation of this new file? Yes, it can be done easily, if the original sequential file is created with holes, which are empty record spaces, as shown in the *Figure 4.3*. Thus, reorganisation of file on addition and update operation can be restricted to only one block, which is read into/ written from the main memory as a single unit. Thus, holes increase the performance of sequential file insertion and deletion. This organisation also supports a concept of overflow area, which can store the spilled over records if a block is full. This technique is also used in index sequential file organisation. A detailed discussion on it can be found in the further readings.

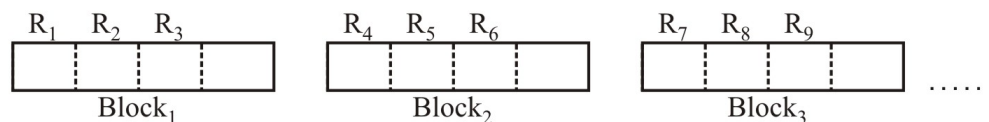


Figure 4.3: A sequential file with empty spaces for record insertions

Advantages of Sequential File Organisation

- It is fast and efficient when dealing with large volumes of data that need to be processed periodically (batch system).

Disadvantages of sequential File Organisation

- Requires all the new transactions to be sorted into a proper sequence for sequential access processing.
- Locating, storing, modifying, deleting, or adding records in the file requires rearranging the file.
- This method is too slow to handle applications requiring immediate updating or responses.

4.4.3 Indexed (Indexed Sequential) File Organisation

It organises the file like a large dictionary, i.e., records are stored in order of the key, but an index is kept which also permits a type of direct access. The records are stored sequentially by primary key values and there is an index built over the primary key field.

To locate a record in a sequential file, which is not indexed sequential, on average, you may read about half the number of records. Therefore, retrieval of a specific record in a sequential file is inefficient, especially if the file has a very large number of disk storage blocks. The use of index in a sequential file improves the query response time by adding an index, which is defined as a set of $\langle \text{index value}, \text{address} \rangle$ pair. An index is a mechanism for faster search, as the size of the index is much smaller than the original file. Thus, an index may be contained entirely in the main memory of the computer.

An indexed sequential file is a sequential file, which is stored in the order of its primary key, that contains an index on its primary key. Indexed sequential file allows advantages of indexing and sequential organization of records. The sequential organisation facilitates sequential processing of records, whereas the index helps in enhancing the performance of locating specific records based on the index value. In order, to make such files more efficient for insertion and deleting of records, such files are supported with overflow blocks. This reduces the need of moving all the records in a file. Figure 4.6 is an example of an indexed sequential file. Please note that the index is represented in Figure 4.6 as $\langle \text{Primary Index Value}, \text{Block Pointer} \rangle$.

4.4.4 Hashed File Organisation

Hashing is the most common form of purely random access to a file or database. It is also used as an optimisation technique to access columns that do not have an index. Hashing involves the use of a hash function. Input to a hash function is the value of the attribute or set of attributes of a record that are to be used for file organisation and the output is the block address or page address, where that record can be found. Figure 4.4 shows a file using hashing file organisation. The hash function used for this file is $key \bmod 4$. Notice that records with different key values can be placed in a Block based on the hashing function. To search the location of a record, you can apply the hashing function on the key value and then search the hashed block. For example, if you are searching for the location of key 29, then the record can be found in $29 \bmod 4 = \text{Block } 1$, read this Block in the main memory and do a linear search on key value to locate the record in the main memory. The most popular form of hashing is division hashing with chained overflow. You can refer to further readings for more details on this file organisation.

Advantages of Hashed File Organisation

1. Insertion or search on hash-key is fast.
2. Best if equality search is needed on hash-key.

Disadvantages of Hashed File Organisation

1. It is a complex file Organisation method.
2. Search is slow.
3. It suffers from disk space overhead.
4. Unbalanced buckets degrade performance.
5. Range search is slow.

ignou
THE PEOPLE'S
UNIVERSITY

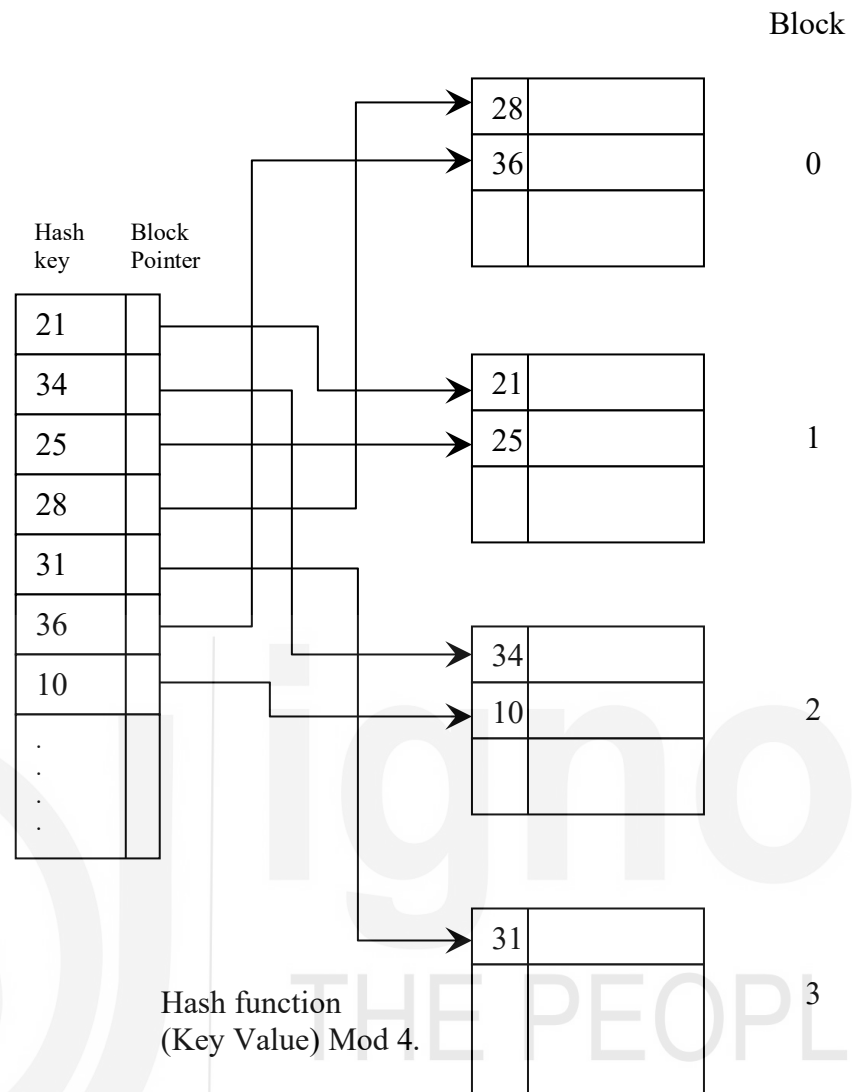


Figure 4.4: A Hashed file

Check Your Progress 1

- 1) List five operations on sequential file Organisation. Comment on the performance of each of these operations.

- 2) What are Direct-Access systems? What can be the various strategies to achieve this?

- 3) What is file organisation and what are the essential factors that are to be considered for a file organisation?

4.5 INDEXES

One of the terms used during the file organisation is the term index. In this section, let us define this term in more detail.

Every printed book that you read, in general, has an index of keywords at the end. Notice that this index is a sorted list of keywords (index values) and page numbers (address) where the keyword can be found. In databases also an index is defined in a similar way, as the *<index value, address>* pair.

The basic advantage of having sorted index pages at the end of the book is that you can locate the description about a desired keyword in the book. You could have used the topic and subtopic listed in the table of contents, but it is not necessary that the given keyword can be found there; also, they are not in any sorted sequence. If a keyword is not listed in index and table of contents, then you need to search each page of the book to find the required keyword, which is very cumbersome. Thus, an index at the back of the book helps in locating the required keyword references very easily in the book.

The same is true for databases that have a very large number of records. A database index allows fast search on the index value in database records. It will be difficult to locate an attribute value in a large database, if the index on that attribute is not provided. In such a case the value is to be searched record-by-record in the entire database, which is cumbersome and time consuming. It is important to note that for a large database all the records cannot be kept in the main memory at a time, thus, data needs to be transferred from the secondary storage device, which is more time consuming.

An index entry consists of a pair consisting of index value and a list of pointers to disk blocks for the records that have that index value. An index contains such information for every stored value of the index attribute. An index file is very small compared to a data file that stores a relation. Also index entries are ordered, so that an index can be searched using an efficient search method like binary search. In case an index file is very large, you can create a multi-level index, that is index on index. Multi-level indexes are defined later in this section.

There are many types of indexes that are categorised as:

Primary index	Single level index	Spare index
Secondary index	Multi-level index	Dense index
Clustering index		

A primary index is defined on the attributes *in the order of which the file is stored*. This field is called the ordering field. A primary index can be on the primary or candidate key of a file. If an index is on the ordering attributes, which are not candidate key attributes, then several records may be related to one ordering field value. This is called clustering index. It is to be noted that there can be only one physical ordering attribute or set of attributes for a file. Thus, a file can have either the primary index or clustering index, not both. Secondary indexes are defined on the non-ordering fields. Thus, there can be several secondary indexes in a file, but only one primary or clustering index.

Primary index

Primary index is a file that contains a sorted sequence of index records having two columns: the ordering key field; and a block address for that key field in the data file. The ordering key field for this index can be the primary key of the data file. Primary index contains one index entry of the ordering key field for each Block of data. An entry in the primary index file contains either the key value of the first record or the key value of the last record, which are stored in that data block; and a pointer to that data block.

Let us discuss the primary index with the help of an example. Let us assume a student database as (Assuming that one block stores only four student records). Figure 4.5 shows a sample of this data file. The sample file is ordered on the attribute - enrolment number.

Block Number	Enrolment Number	Name	City	Programme
BLOCK 1	2109348	...	CHENNAI	CIC
	2109349	...	CALCUTTA	MCA
	2109351	...	KOCHI	BCA
	2109352	...	KOCHI	CIC
BLOCK 2	2109353	...	VARANASI	MBA
	2238389	...	NEW DELHI	MBA
	2238390	...	VARANASI	MCA
	2238411	...	NEW DELHI	BCA
BLOCK 3	2238412	...	AJMER	MCA
	2238414	...	NEW DELHI	MCA
	2238422	...	MUMBAI	BSC
	2258014	...	MUMBAI	BCA
BLOCK 4	2258015	...	MUMBAI	BCA
	2258017	...	NEW DELHI	BSC
	2258018	...	MUMBAI	MCA
	2258019	...	LUCKNOW	MBA
...	...	...	...	...
BLOCK r	2258616	...	AJMER	BCA
	2258617	...	LUCKNOW	MCA
	2258618	...	NEW DELHI	BSC
	2318935	...	FARIDABAD	MBA
...	...	...	...	...
BLOCK N-1	2401407	...	BAREILLY	CIC
	2401408	...	BAREILLY	BSC
	2401409	...	AURANGABAD	BCA
	2401623	...	NEW DELHI	MCA
BLOCK N	2401666	...	MUMBAI	MCA
	2409216	...	LUCKNOW	MBA
	2409217	...	ALMORA	BCA
	2409422	...	MUMBAI	BSC

Figure 4.5: A Student file stored in the order of student enrolment numbers

The primary index on this file would be on the ordering field – enrolment number. The primary index on this file is shown in Figure 4.6. Please note the following points in Figure 4.6.

- An index entry is defined as the attribute value, pointer to the block where that record is stored. The pointer physically is represented as the binary address of the block.
- Since there are four student records, which of the key values should be stored as the index value? We have used the first key value stored in the block as the

index key value. This is also called the anchor value. All the records stored in the given block have ordering attribute value as the same or more than this anchor value.

- The primary index may be smaller in size, as it contains one index entry for each storage data block. Also notice that not all the records need to have an entry in the index file. This type of index is called a non-dense index. Thus, the primary index is non-dense index.
- To locate the record of a student whose enrolment number is 2238422, you need to find two consecutive entries of indexes such that index value $1 < 2238422 < \text{index value } 2$. In the Figure 4.6, you can find the third and fourth index values as: 2238412 and 2258015 respectively satisfying the properties as above. Thus, the required student record must be found in Block 3.

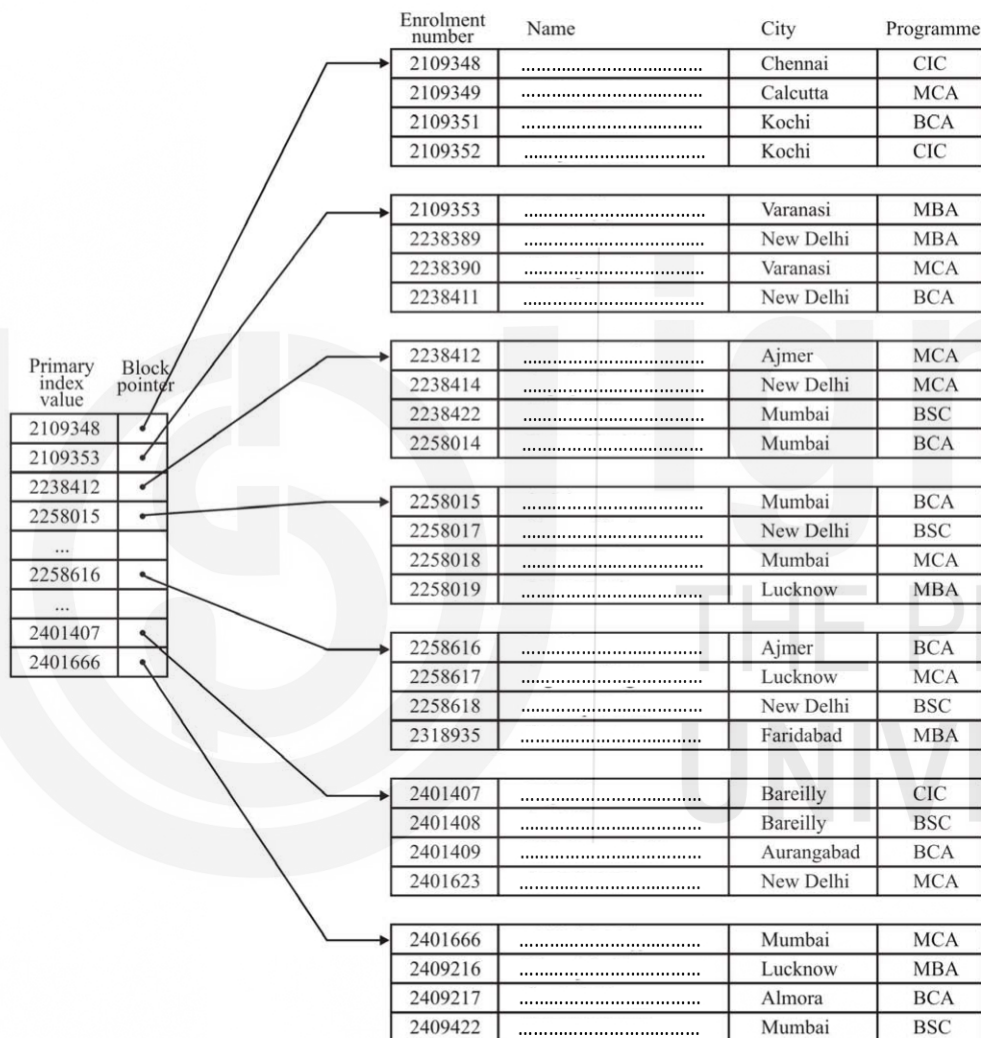


Figure 4.6: A Student file and the Primary Index on Enrolment Number

But does primary index enhance efficiency of searching? Let us explain this with the help of an example (Please note we will define savings in terms of the number of block transfers, as that is the most time-consuming operation during searching).

Example 1: An ordered student file (ordering field is enrolment number) has 20,000 records stored on a disk having the Block size as 1 K. Assume that each student record is of 100 bytes, the ordering field is of 8 bytes, and block pointer is also of 8 bytes, find how many block accesses on average may be saved on using primary index.

Answer:

Number of accesses without using Primary Index:

Number of records in the file = 20000

Block size = 1024 bytes

Record size = 100 bytes

Number of records per block = integer value of $[1024 / 100] = 10$

Number of disk blocks acquired by the file

= $[\text{Number of records} / \text{records per block}]$

= $[20000/10] = 2000$

Assuming a block level binary search, it would require $\log_2 2000$

= about 11 block accesses.

Number of accesses with Primary Index:

Size of an index entry = $8+8 = 16$ bytes

Number of index entries that can be stored per block

= integer value of $[1024 / 16] = 64$

Number of index entries = number of disk blocks = 2000

Number of index blocks = ceiling of $[2000/ 64] = 32$

Number of index block transfers to find the value in index blocks = $\log_2 32 = 5$

One block transfer will be required to get the data records using the index pointer after the required index value has been located.

So total number of block transfers with primary index = $5 + 1 = 6$.

Thus, the Primary index would save $11 - 6 = 5$ block transfers for the given size of data and index.

Is there any disadvantage of using a primary index? Yes, a primary index requires the data file to be ordered, this causes problems during insertion and deletion of records in the file. This problem can be taken care of by selecting a suitable file organisation that allows logical ordering only.

Clustering Indexes.

It may be a good idea to keep records of the students in the order of the programme they have registered, as most of the data file accesses may require programme wise student data. A file can be ordered and physically stored on non-key attributes; an index that is created on such non-key attributes would have multiple records pointed to by a single index entry. Such an index is called a clustering index. *Figure 4.7 and Figure 4.8* show the clustering indexes in the same file organised in different ways.

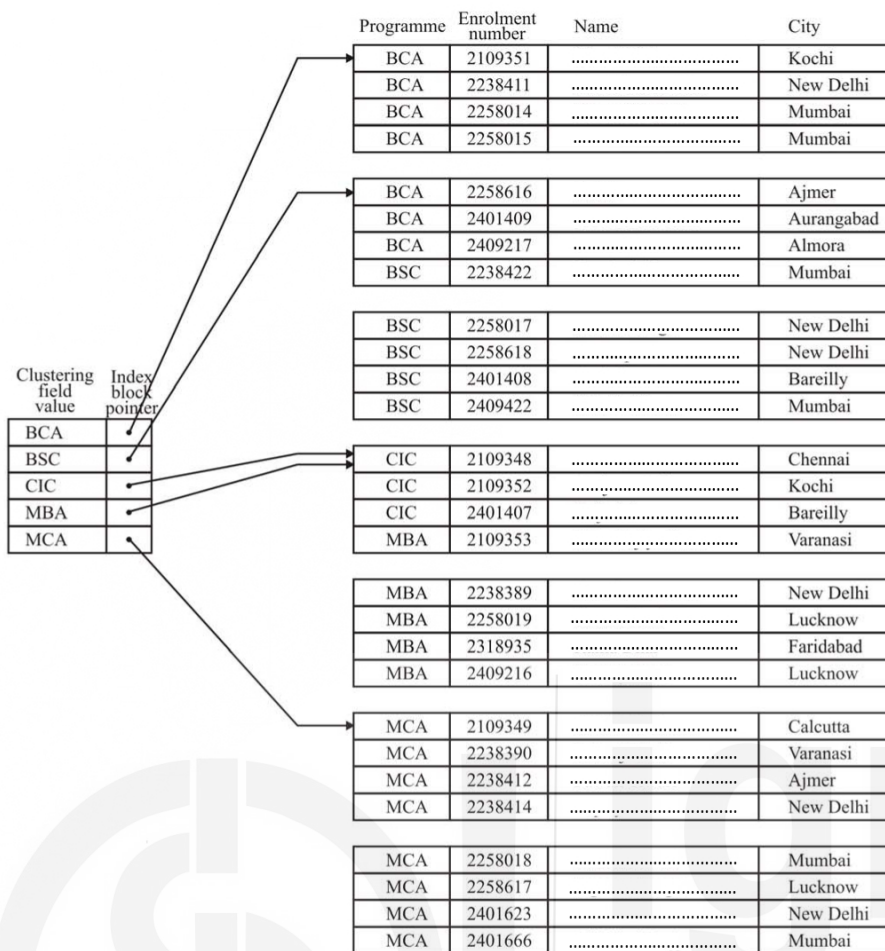


Figure 4.7: A clustering Index on Programme in the Student file

Please note the following points about the clustering index as shown in the *Figure 4.7*

- The clustering index is an ordered file having the clustering index value and a block pointer to the first block where that clustering field value first appears.
- Clustering index is also a sparse index. The size of the clustering index is smaller than the primary index as far as the number of entries is concerned.

Please note that in *Figure 4.7*, the data file can have a single block in which data of students of multiple programmes are stored. You can improve upon this organisation by allowing only one Programme data in one block. Such an organisation and its clustering index is shown in the *Figure 4.8*:

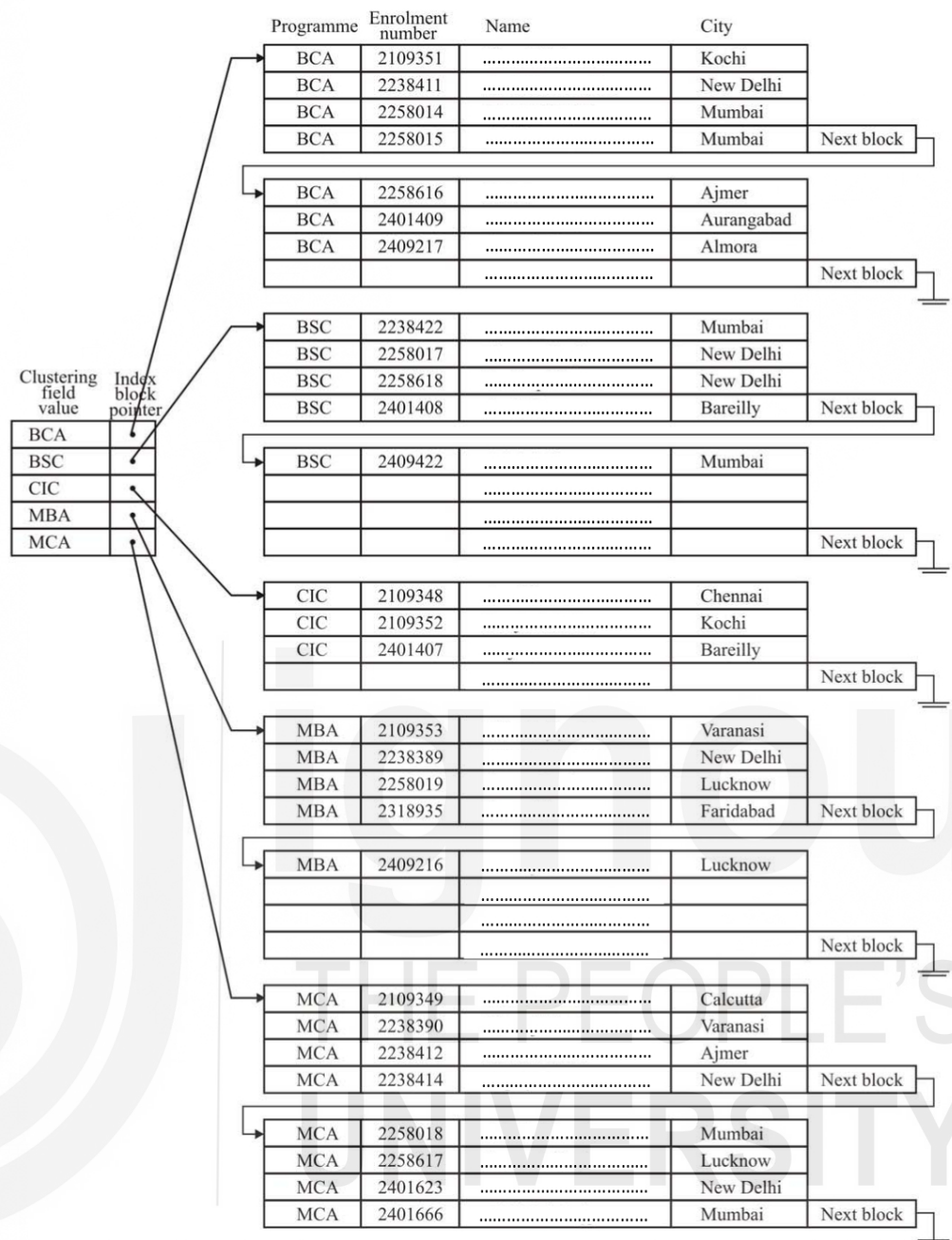


Figure 4.8: Clustering index with separate blocks for each clustering attribute value

Please note the following points for the above:

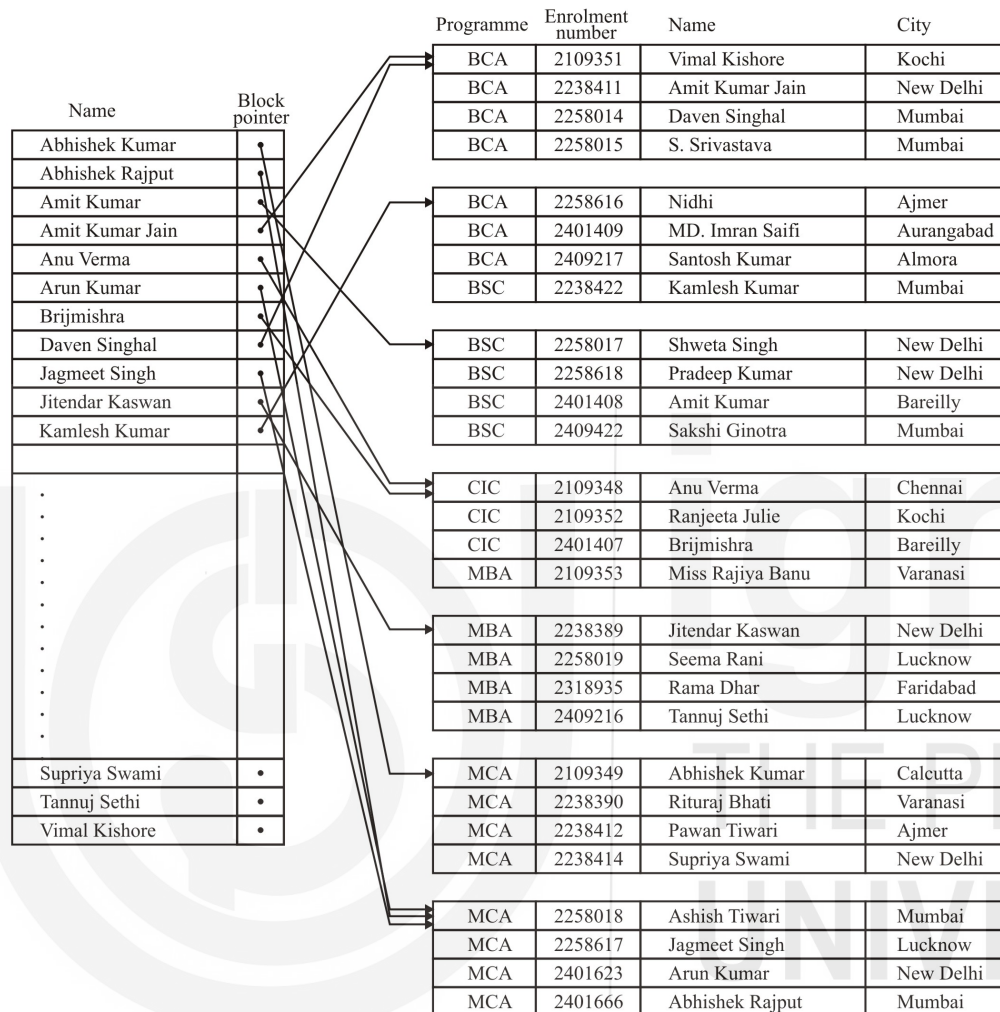
- Data insertion and deletion is easier than in the earlier clustering files, yet even now it is cumbersome.
- The blocks allocated for one index entry are in the form of linked list blocks.
- Clustering index is another example of a non-dense index as it has one entry for every distinct value of the clustering index attribute and not for every record in the file.

Secondary Indexes

Consider the student database and its primary and clustering index (only one will be applicable at a time). Now consider the situation when the database is to be searched or accessed in the alphabetical order of student names. Any search on a student name would require a sequential data file search, as there is no ordering on student names. Thus, searching for a student's name is going to be very time consuming. Such a search on an average would require reading half of the total number of blocks. Thus, you need secondary indices in database systems. A secondary index is a file that contains records containing a secondary index field value which is not the ordering field of the data file, and a pointer to the block that contains the data record. Please

note that although a data file can have only one primary index (as there can be only one ordering of a database file), it can have many secondary indices.

Secondary indexes can be defined on an alternate key or non-key attributes. A secondary index that is defined on the alternate key will be dense, while a secondary index on non-key attributes would require a bucket of pointers for one index entry. Let us explain them in more detail with the help of *Figure 4.9*.



**Figure 4.9: A Dense Secondary Index on a non-ordering key field of a file
(The file is organised on the clustering field “Programme”)**

Please note the following in *Figure 4.9*.

- The names in the data file are unique and thus are being assumed as the alternate key. Each name therefore is appearing as the secondary index entry.
- The pointers are block pointers, thus are pointing to the beginning of the block and not a record. For simplicity, we have not shown all the pointers in Figure 4.9.
- This type of secondary index file is dense index as it contains one entry for each record/distinct value.
- The secondary index is larger than the Primary index as we cannot use block anchor values here as the secondary index attributes are not the ordering attribute of the data file.
- To search a value in a data file using name, first the index file is (binary) searched to determine the block, where the record having the desired key value

can be found. Then this block is transferred to the main memory where the desired record is searched and accessed.

- A secondary index file, usually, has a larger number of index entries than that of primary index. However, the secondary index improves the search time to a greater proportion than that of a primary index. This is due to the reason - If a primary index does not exist even then, you can perform binary search on the blocks of data records, as the records are ordered in the sequence of primary index value. However, if a secondary key does not exist, then you may need to search the records sequentially. This fact is demonstrated with the help of Example 2.

Example 2: Let us reconsider the problem of Example 1 with a few changes. An unordered student file, which is not ordered on a primary key, has 20,000 records stored on a disk having a Block size as 1 K. Assume that each student record is of 100 bytes, the secondary index field is of 8 bytes, and the block pointer is also of 8 bytes, find how many block accesses on average may be saved on using a secondary index on enrolment number.

Answer:

Number of accesses without using a Secondary Index:

Number of records in the file = 20000

Block size = 1024 bytes

Record size = 100 bytes

Number of records per block = integer value of $[1024 / 100] = 10$

Number of disk blocks acquired by the file

= $[\text{Number of records} / \text{records per block}]$

= $[20000/10] = 2000$

Since the file is un-ordered any search on an average will require about half of the above blocks to be accessed. Thus, average number of block accesses = 1000

Number of accesses with Secondary Index:

Size of an index entry = $8+8 = 16$ bytes

Number of index entries that can be stored per block

= integer value of $[1024 / 16] = 64$

Number of index entries = number of records = 20000

Number of index blocks = ceiling of $[20000/ 64] = 320$

Number of index block transfers to find the value in index blocks

= ceiling of $[\log_2 320] = 9$

One block transfer will be required to get the data records using the index pointer after the required index value has been located. So total number of block transfers with secondary index = $9 + 1 = 10$

Thus, the Secondary index would save about 1990 block transfers for the given case. This is a huge saving compared to a primary index. Please also compare the size of the secondary index to the primary index.

Let us now see an example of a secondary index that is on an attribute that is not an alternate key.

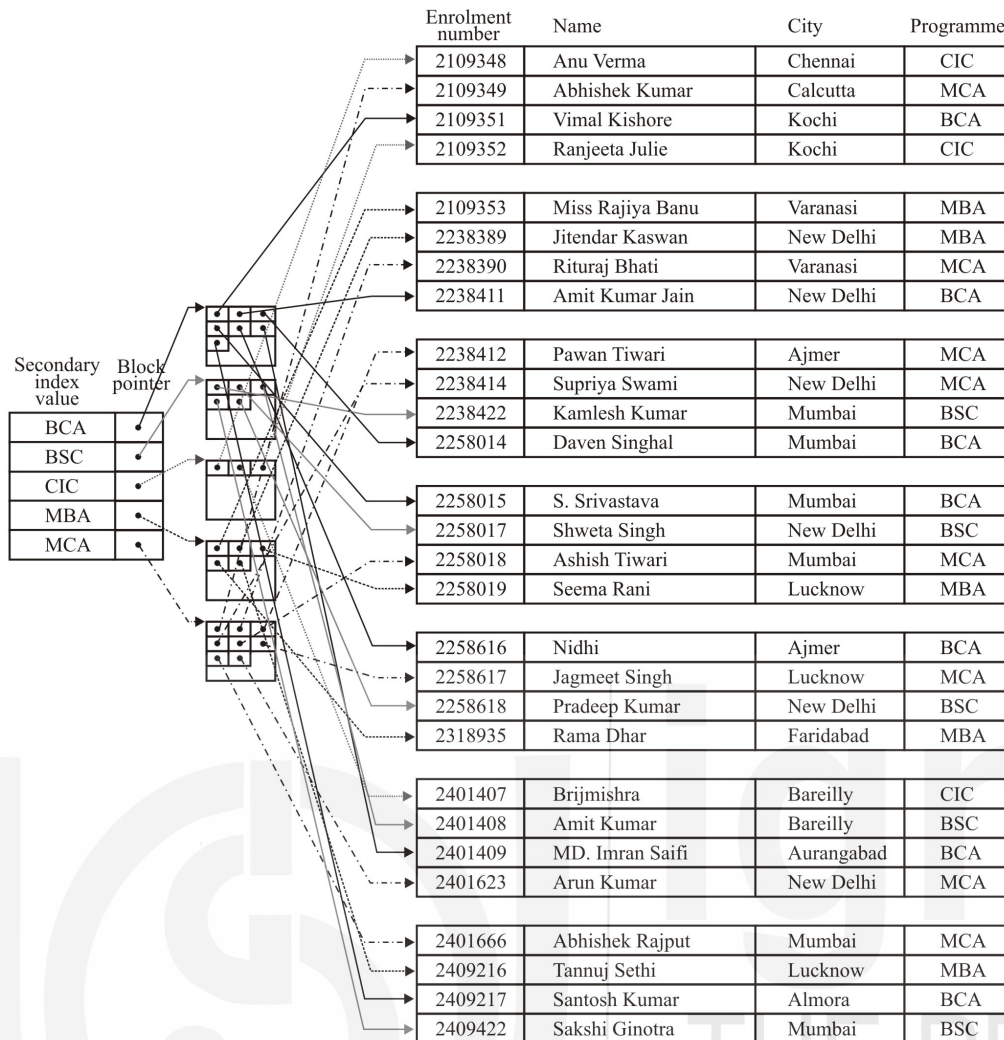


Figure 4.10: An example of Secondary Index with single level of indirection
 (Index entries are of fixed length and have unique field values)
 (The file is organised on the primary key)

A secondary index that needs to be created on a field that is not a candidate key can be implemented using several ways. We have shown here the way in which a block of pointer records is kept for implementing such an index. This method allows the index entries to be of fixed length. It also allows only a single entry for the value of the indexing attribute. In addition, the level of indirection allows multiple index pointers to be stored in a single block of data. The algorithms for searching the index, inserting and deleting new values into an index are very simple in such a scheme. Thus, this is the most popular scheme for implementing such secondary indexes.

Sparse and Dense Indexes

As discussed earlier, an index is defined as the ordered $\langle \text{index value}, \text{address} \rangle$ pair. These indexes in principle are the same as that of indexes used at the back of the book. The key ideas of the indexes are:

- They are sorted on the order of the index value (ascending or descending) as per the choice of the creator.
- The indexes are logically separate files (just like separate index pages of the book).
- An index is primarily created for fast access to information.
- The primary index is the index on the ordering field of the data file, whereas a secondary index is the index on any other field, thus, is more useful.

But what are sparse and dense indexes?

A dense index contains one index entry for every value of the indexing attributes, whereas a sparse index also called non-dense index contains few index entries out of the available indexing attribute values. For example, the primary index on enrolment number is sparse, while secondary index on student name is dense.

Multilevel Indexing Scheme

For small files, the indexing scheme keeps the address of the block file in each index entry. Such indices would be small and can be processed efficiently in the main memory. However, for a large file the size of the index can also be very large. In such a case, you can create indexes at several levels, with the last level pointing to the data records. Figure 4.11 shows this scheme.

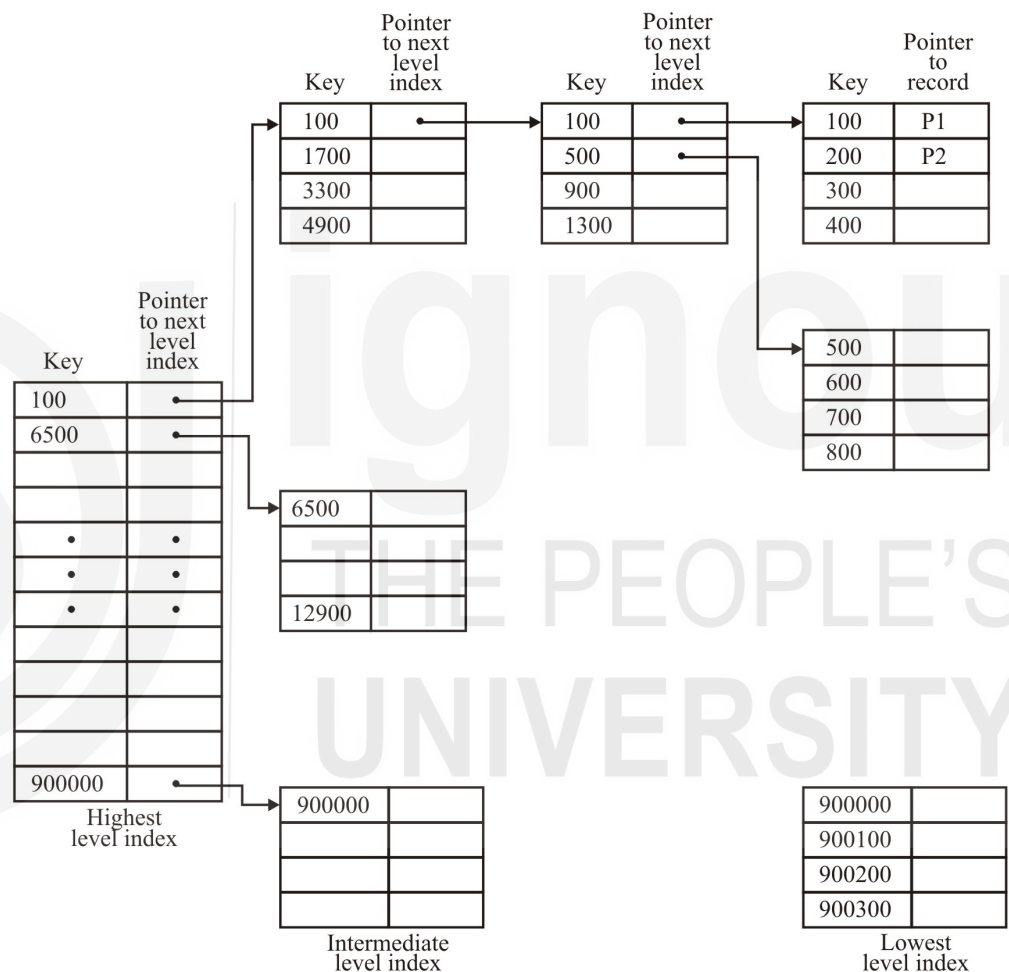


Figure 4.11: Hierarchy of Indexes

Please note the following relating to multi-level indexes:

- The lowest level index, as shown in Figure 4.11, has one entry for each record, though in a different index block. Thus, the lowest level index occupies the maximum space.
- Thus, maintenance of the multi-level indexes is expensive, as multiple indexes are to be maintained for every insertion and deletion of records.

After discussing so much about the indexes, let us now turn our attention to how an index can be implemented in a database. The indexes are implemented through B-Tree. The following section examines the index implementation in more detail.

Check Your Progress 2

- 1) A file contains 40,000 student records of 200 bytes, each having the 1 KB as the size of a block. What would be the size of the primary index, if the size of the

primary key is 4 byte and the block address is 8 bytes? What would be the average number of block accesses to access a record using this Index?

.....

- 2) What would be the size of secondary index for the student file as stated in question 1 if it has a secondary index on its alternate key of size 8 bytes. What would be the average number of block accesses to access a record using this secondary Index?

.....

.....

.....

- 3) Which of the following indexes are dense indexes? Give reasons.
 (i) Primary Index (ii) Clustering Index (iii) Secondary Index on alternative key
 (iv) Secondary index on non-key attributes with unique attribute values

.....

.....

.....

.....

4.6 IMPLEMENTING INDEX USING TREE STRUCTURE

Let us discuss the data structure that is used for creating indexes.

Can we use Binary Search Tree (BST) as Indexes?

Let us first reconsider the binary search tree. A BST is a data structure that has a property that all the keys that are to the left of a node are smaller than the key value of the node and all the keys to the right are larger than the key value of the node.

To search a typical key value, you start from the root and move towards left or right depending on the value of the key that is being searched. Since an index is a *<value, address>* pair, thus while using BST, you need to use the value as the key and address field must also be specified in order to locate the records in the file that is stored on the secondary storage devices. The following figure demonstrates the use of BST index for a University where a dense index exists on the enrolment number field.

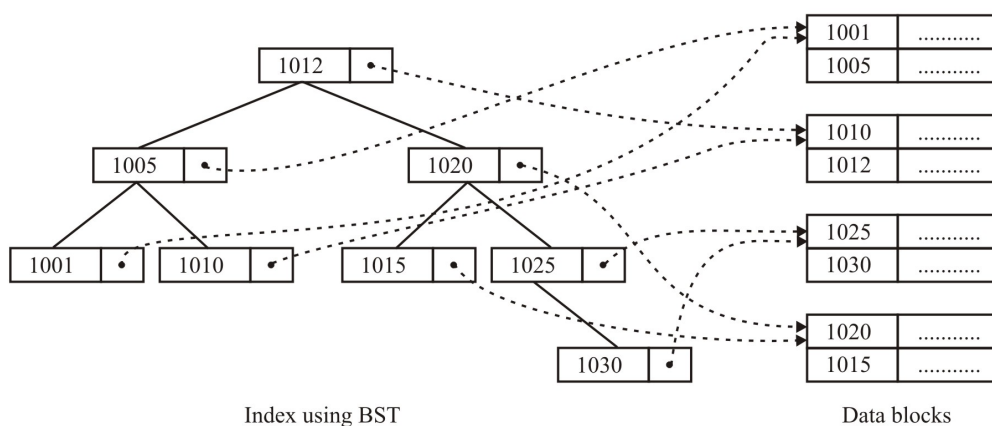


Figure 4.12: The Index structure using Binary Search Tree

Please note in Figure 4.12 that a key value is associated with a pointer to a record. A record consists of the key value and other information fields. Please note that a node on BST stores the $\langle \text{key value}, \text{address} \rangle$ pair.

Now, let us examine the suitability of BST as a data structure to implement indexes. A BST as a data structure is suitable for an index if the complete index is contained in the primary memory. However, indexes are quite large in nature and require a combination of primary and secondary storage. Therefore, you can use B-Tree data structure to implement the index.

A B-Tree as an index has two advantages:

- It is completely balanced.
- Each node of B-Tree can have a number of keys. An ideal node size of B-Tree would be equal to the block size of the secondary storage device that is being used for storing the index.

The question that needs to be answered here is: what should be the order of B-Tree for an index? The suggested order is from 80-200 depending on various index structures and block size.

B-tree is a data structure, which was proposed by R. Bayer and E. McCreight of Bell Scientific Research Labs in 1970. The B-Tree and its variants are secondary storage structures and have been found to be very useful for implementing indexes. An N order B-tree has:

- A node of B-tree of order N can have children/paths in the range - ceiling of $[N/2]$ to N. However, the root node of the tree can have 2 to N children/paths.
- Each node can have one fewer key than the number of children/paths, but a maximum of N-1 keys can be stored in a node.
- The keys are normally arranged in a node in an increasing order.
- If a new key is inserted into a full node of order N (i.e. it already contains N-1 keys), then on addition of this new key value, the node would have N+1 paths (N keys). This node is split into two nodes and the median key value is moved to the parent of this node. In case the node that is being split is the root node, then it is split into two nodes and a new root node is created by using the median key of the node being split.
- B-tree does not allow any empty sub-tree, therefore, all the leaves of B-tree are at the same level. Therefore, a B-tree is a completely balanced tree.

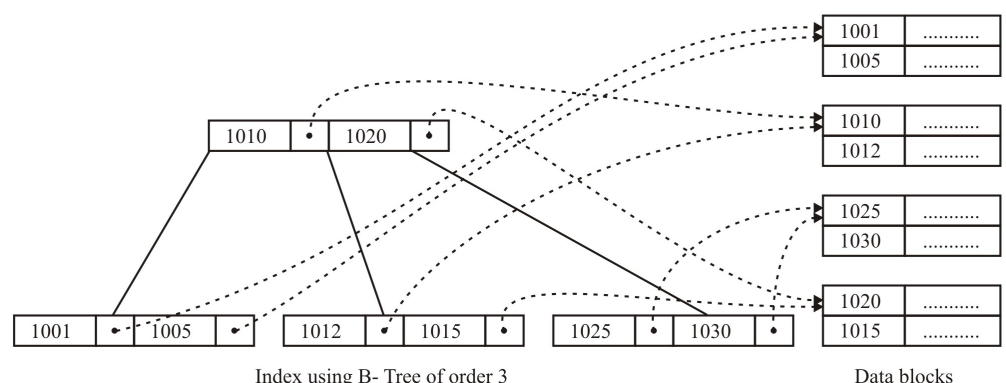


Figure 4.13: A B-Tree as an index

A B-Tree index is shown in Figure 4.13. The B-Tree has a very useful variant called B+Tree, which has all the key values at the leaf level also, in addition to the higher level. For example, the key value 1010 in Figure 12 will also exist at leaf level. In

addition, these lowest level leaves are linked through pointers. Thus, the B+tree is a very useful structure for index-sequential organisation. You can refer to further readings for more details on these topics.

4.7 MULTI-KEY FILE ORGANISATIONS

Till now we have discussed file organisations having the single access key. But is it possible to have file organisations that allow access of records on more than one key field? This section discusses the two file organisations that allow multiple access paths, with each path having a different key. These are called multi-key file Organisations. These file organisations, in general, are part of a real database management system. Two of the commonest techniques for this Organisation are:

- Multi-list file Organisation
- Inverted file Organisation

Let us discuss these techniques in more detail. But first let us discuss the need for the Multiple access paths.

4.7.1 Multiple Access Paths

In practice, most of the online information systems require the support of multi-key files. For example, consider a banking database application having many kinds of users such as:

- Teller
- Loan officers
- Branch manager
- Account holders

All these users access the bank data however in a different way. Let us assume a sample data format for the Account relation in a bank as:

Account Relation:

Account Number	Account Holder Name	Branch Code	Account type	Balance	Permissible Loan Limit

A teller may access the record above to check the balance at the time of withdrawal. S/he needs to access the account based on branch code and account number. A loan approver may be interested in finding the potential customer by accessing the records in decreasing order of permissible loan limits. A branch manager may need to find the top ten most preferred customers in each category of account, so s/he may access the database in the order of account type and balance. The account holder may be interested in her/his own record. Thus, all these applications are trying to refer to the same data but using different key values. Thus, all the applications as above require the database file to be accessed in different format and order.

Multiple indexes can be used to access a data file through multiple access paths. In such a scheme only one copy of the data is kept, only the number of paths is added with the help of indexes. Let us discuss two important approaches, viz. multi-list file organisation and Inverted file organisation.

4.7.2 Multi-list file Organisation

This file organisation, as the name suggests, consists of multiple lists or indexes. The records in each list are linked from the index value. The linking of records, in general, is done in the sorted sequence of the key attribute to facilitate searching, insertion and deletion operations. The following example explains the multi-list file organisation.

A sample data of employees of an organisation is given in *Figure 4.14*. Assume that the Empid is the key attribute. You can create multiple index lists using this data.

Assumed Record Number	Employee id (Empid)	Employee Name	Job Title	Highest Qualification	Gender (Female F /Male M)	City of posting	Married - M/ Single - S	Salary per month
A	795	Praveen	Engineer	B. Tech.	M	Dehradun	S	16,200/-
B	495	Rohini	Manager	B. Tech.	F	Dehradun	M	19,000/-
C	905	Rishika	Manager	MCA	F	Jaipur	S	17,100/-
D	705	Gaurav	Engineer	B. Tech.	M	Jaipur	M	13,200/-
E	595	Dipti	Manager	MCA	F	Jaipur	S	14,100/-

Figure 4.14: Sample Employee Data

The primary link order (in the order of primary key Empid) would be:

B(495), E(595), D(705), A(795), C(905)

The primary index for this file would be:

≥ 500 but < 700
 ≥ 700 but < 900
 ≥ 900 but < 1100

The index file for the example data as per Figure 4.14 is shown in *Figure 4.15*.

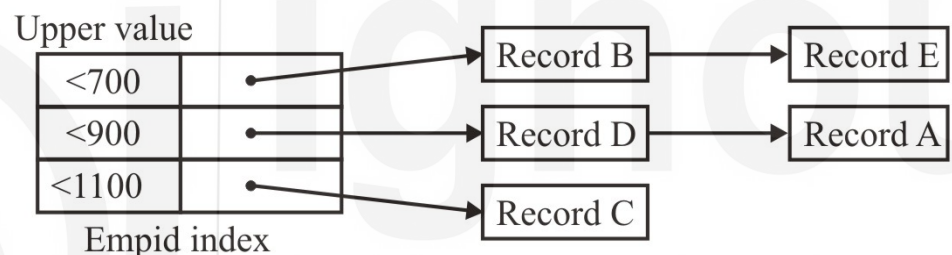


Figure 4.15: Linking together all the records in the same index value.

Please note that in the *Figure 4.15*, those records that fall in the same index value range of Empid are linked together. These lists are smaller than the total range, which will improve search performance.

This file can be supported by many more indexes that will enhance the search performance on various fields, thus, creating a multi-list file organisation. *Figure 4.16* shows various indexes and lists corresponding to those indexes. For simplicity we have just shown the links and not the complete record. Please note that nodes in the original file are assumed to be in the order of Empid's.

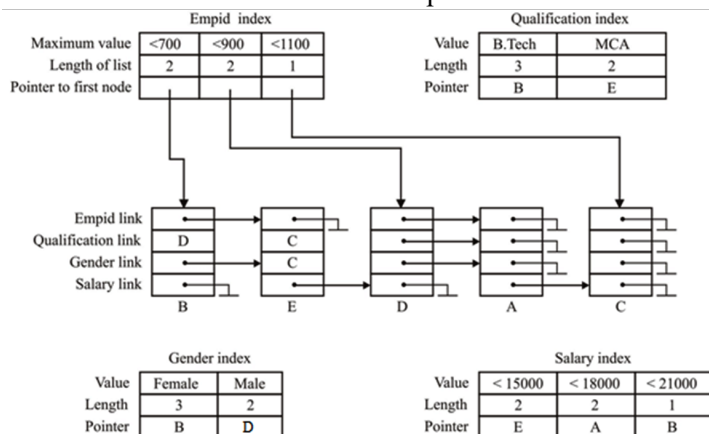


Figure 4.16: Multi-list representation for Figure 4.14

In *Figure 4.16*, the qualification index can be read as:

B. Tech. Starts at B → D → A.

MCA Starts at E → C.
 Gender Index:
 Female: Starts at B, → E, → C.
 Male: Starts at B, → D, → A.

An interesting addition that can be done in the indexing scheme of multi-list organisation is that an index entry may contain the length of each sub-list and the index entry should have a pointer to the first record of that list. The length information is useful when the query contains a Boolean expression. For example, if you need to find the list of Female employees who have MCA qualifications, you can find the results in two ways. Either you go to the Gender index and search the Female index list for MCA qualification, or you search the qualification index to find MCA list and in MCA list search for Female candidates. Since the size of MCA list is 2 while the size of Female list is 3 so the preferable search will be through a smaller MCA index list. Thus, the information about the length of the list may help in reducing the search time in the complex queries. Performance of this structure is not good under heavy insertion and deletion of records. However, it is a good structure for searching records in case the appropriate index exists.

4.7.3 Inverted File Organisation

Inverted file organisation is one file organisation where the index structure is most important. In this organisation the basic structure of file records does not matter much. This file organisation is somewhat similar to that of multi-list file organisation with the key difference that in multi-list file organisation index points to a list, whereas in inverted file organisation the index itself contains the list. Thus, maintaining the proper index through proper structures is an important issue in the design of inverted file organisation. Let us show inverted file organisation with the help of data given in *Figure 4.14*.

Let us assume that the inverted file organisation for the data shown contains a dense index. *Figure 4.17* shows how the data can be represented using inverted file organisation.

Empid index		Qualification index		Salary index	
495	B	B.Tech	B, D, A	13200	D
595	E	MCA	E, C	14100	E
705	D	Gender index		16200	A
795	A			17100	C
905	C			19000	B
		Female	B, E, C		
		Male	D, A		

Figure 4.17: Some of the indexes for fully inverted file

Please note the following points for the inverted file organisation:

- The index entries are of variable lengths as the number of records with the same key value is changing, thus, maintenance of index is more complex than that of multi-list file organisation.
- The queries that involve Boolean expressions require accesses only for those records that satisfy the query in addition to the block accesses needed for the indices. For example, the query about Female, MCA employees can be solved by the Gender and Qualification index. You just need to take the intersection of record numbers on the two indices. (Please refer to *Figure 4.17*). Thus, any complex query requiring Boolean expression can be handled easily through the help of indices.

4.8 IMPORTANCE OF FILE ORGANISATION IN DATABASES

To implement a database efficiently, there are several design tradeoffs needed. One of the most important ones is the file Organisation. For example, if there were to be an application that required only sequential batch processing, then the use of indexing techniques would be pointless and wasteful.

There are several important consequences of an inappropriate file Organisation being used in a database. The wrong file Organisation will result in:

- much larger processing time for retrieving or modifying the required record.
- undue disk access that could stress the hardware.

Needless to say, there could be many undesirable consequences at the user level, such as making some applications impractical.

Check Your Progress 3

- 1) What is the difference if indexes are implemented using binary search tree or using B-tree?

.....
.....

- 2) What are the advantages of using B+ tree index over B tree index if the supported file organisation is required to access the records sequentially too.

.....
.....
.....

- 3) What is the need of a multi-list organisation? What is the advantage of storing the number of records in an index entry?

.....
.....
.....

4.9 SUMMARY

In this unit, we discussed the physical database design issues in which we had addressed the file Organisation and file access method. The unit also discusses different types of file organization giving their advantages and their disadvantages.

An index is an important component of a database system, as one of the key requirements of DBMS is efficient access to data. This unit explains various types of indexes that may exist in database systems. Some of these are: Primary index, clustering index and secondary index. The secondary index results in better search performance but adds on the task of index updating. This unit also discusses two multi-key file organisations viz. multi-list and inverted file organisations. These are very useful for improving query performance.

4.10 SOLUTIONS / ANSWERS

Check Your Progress 1

1)

Operation	Comments
File Creation	It will be efficient if transaction records are ordered by record key
Record Location	As it follows the sequential approach it is inefficient. On an average, half of the records in the file must be processed to locate a record.
Record Creation	It will require you to browse through all the records to check if such a record already exists or not. Thus, the entire file must be read and written. Efficiency improves if a group of records are created together. This operation could be combined with deletion and modification transactions to achieve greater efficiency.
Record Deletion	The entire file must be read and written. Efficiency improves with greater number of deletions. This operation could be combined with addition and modification transactions to achieve greater efficiency.
Record Modification	Very efficient if the number of records to be modified is high and the records in the transaction file are ordered by the record key.

- 2) Direct-access systems do not search the entire file; rather they move directly to the record, which is to be accessed. To be able to achieve this, several strategies like relative addressing, hashing and indexing can be used.
- 3) It is a technique for physically arranging the records of a file on secondary storage devices. When choosing the file Organisation, you should consider the following factors:
1. Data retrieval speed
 2. Data processing speed
 3. Efficient use of storage space
 4. Protection from failures and data loss
 5. Scalability
 6. Security

Check Your Progress 2

1) **Number of accesses without using Primary Index:**

Number of records in the file = 40000

Block size = 1024 bytes

Record size = 200 bytes

Number of records per block = integer value of $[1024 / 200] = 05$

Number of disk blocks acquired by the file

= $[\text{Number of records} / \text{records per block}]$

= $[40000/05] = 8000$

The file is in the order of primary index, so binary search can be employed to access the file. As $2^{13} = 8192$. Thus, about 13 Block accesses would be required to locate the block containing the desired record.

Number of accesses with Primary Index:

Size of an index entry = $4+8 = 12$ bytes

Number of index entries that can be stored per block
= integer value of $[1024 / 12] = 85$

Number of index entries = number of disk Blocks of file = 8000

Number of index blocks = ceiling of $[8000 / 85] = 94$

Number of index block transfers to find the value in index blocks
= ceiling of $[\log_2 94] = 7$

One block transfer will be required to get the data records using the index pointer after the required index value has been located. So total number of block transfers with secondary index = $7 + 1 = 8$

Thus, the Primary index would save about 5 block transfers for the given case.

2) **Number of accesses without using Secondary Index:**

Number of disk Blocks = 8000 (as computed in Question 1)

The file is in the order of primary index, so search on alternative key would require on an average half of the Blocks to be searched.

Average number of block transfers (without secondary index) = 4000

Number of accesses with Secondary Index on alternate key (dense index):

Size of an index entry = $8+8 = 16$ bytes

Number of index entries that can be stored per block
= integer value of $[1024 / 16] = 64$

Number of index entries = number of records = 40000

Number of index blocks = ceiling of $[40000 / 64] = 625$

Number of index block transfers to find the value in index blocks
= ceiling of $[\log_2 625] = 10$

One block transfer will be required to get the data records using the index pointer after the required index value has been located. So total number of block transfers with secondary index = $10 + 1 = 11$

Thus, the Secondary index would save a large number of block transfers.

- 3)
- (i) Sparse as only one entry per Block of data.
 - (ii) Dense, as one index entry for each attribute value, sparse if multi-level index is used.
 - (iii) Dense, as one index entry for each attribute value
 - (iv) Dense, as one index entry for each unique attribute

Check Your Progress 3

- 1) In a B+ tree the leaves are linked together to form a sequence set; interior nodes exist only for the purposes of indexing the sequence set (not to index into data/records). The insertion and deletion algorithms differ slightly.
- 2) Sequential access to the keys of a B-tree is much slower than sequential access to the keys of a B+ tree, since the latter have all the keys linked in sequential order in the leaf.
- 3) The multi-list organisation enhances the query answering capability of databases on multiple key values. The number of records in an index entry determines the length of the index on a specific attribute value. This enhances the performance of searching when more than one search term are used, as you can select the smaller list and then apply second search term on it.